

# Rúbrica de evaluación

Bloque React y Proyecto Final Integrador · Escala vigesimal (0 a 20)

## 1. Proyecto final: tema libre, requisitos mínimos

El proyecto final es una aplicación web individual, de **tema libre** (comercio, catálogo de películas, recetario, biblioteca, gestión de eventos, entre otros). Independientemente del tema, la aplicación debe incluir los siguientes elementos, que corresponden al temario del bloque:

- Una **entidad principal** listada desde una **API real** (pública o simulada: DummyJSON, TMDB, PokeAPI, etc.).
- Una **colección del usuario** sobre esa entidad (carrito, favoritos, lista de lectura, reservas), con persistencia.
- Un **formulario principal** con validación (checkout, registro, publicación).
- **Sesión de usuario** con áreas protegidas.
- Aplicación **desplegada** y operativa.

## 2. Componentes de la nota (20 pts)

| COMPONENTE                                              | PESO                | FORMA DE MEDICIÓN                                                                                                                                                                                                                                                                                                                                                                                 |
|---------------------------------------------------------|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Proceso progresivo</b><br>avance clase a clase       | <b>3 pts</b> (15%)  | Se evalúa el avance registrado en el repositorio después de cada clase: réplica de la sesión completa y <code>npm run build</code> sin errores. Puntaje por sesión: 2 = réplica completa y tareas intentadas · 1 = solo la réplica · 0 = sin entrega. Se promedian todas las sesiones y se escala a 3. El historial de commits, con sus fechas, acredita la autoría y la continuidad del trabajo. |
| <b>Proyecto final</b><br>producto desplegado            | <b>11 pts</b> (55%) | Según la rúbrica de la sección 4 (12 temas del temario, calificados con la escala de la sección 3) y el bonus de arquitectura de la sección 5.                                                                                                                                                                                                                                                    |
| <b>Sustentación</b><br>semanas de presentación          | <b>3 pts</b> (15%)  | Exposición de 10 minutos (demostración de la aplicación y defensa de una decisión de diseño propia), seguida de una ronda de preguntas de 5 minutos. Total: 15 minutos por estudiante.                                                                                                                                                                                                            |
| <b>Dominio conceptual</b><br>comprensión de fundamentos | <b>3 pts</b> (15%)  | Dos o tres preguntas seleccionadas al azar del banco conceptual (sección 7), respondidas sobre el propio código durante la ronda de preguntas.                                                                                                                                                                                                                                                    |

**Requisitos de recepción del proyecto final.** No otorgan puntaje; su incumplimiento impide la corrección:

- `npm run build` termina **sin errores**.
- La aplicación está **desplegada en Vercel** y operativa desde cualquier dispositivo.
- La consola del navegador se mantiene **sin errores** en el flujo principal.

## 3. Escala de calificación por tema

Cada tema de la sección 4 se califica con la siguiente escala. Los fundamentos (HTML semántico, CSS/Tailwind, responsive, accesibilidad) no se califican por separado: en React, el JSX es el HTML de la aplicación y Tailwind su CSS, por lo que se evalúan **en el tema donde aparecen**. Un formulario sin labels descuenta en Formularios; un listado que falla en móvil descuenta en Renderizado de listas.

### Logrado · 100%

Funciona de extremo a extremo, el código sigue las prácticas del curso y la construcción es correcta: semántica, responsive, accesible, lógica limpia.

### Parcial · 60%

Funciona, pero con deficiencias advertidas explícitamente durante el curso.

### Insuficiente · 20%

Presente pero no funcional, o con evidencia de falta de comprensión.

### Ausente · 0%

No se implementó.

**Criterio acotado para TypeScript.** El plan de estudios incluyó una sola sesión de TypeScript, por lo que se exige únicamente: interfaces del dominio del proyecto (la entidad principal, el ítem de la colección) utilizadas efectivamente, y props tipadas en los componentes. No se exigen genéricos propios ni narrowing avanzado.

#### 4. Rúbrica del proyecto final (11 pts): los 12 temas del temario

| TEMA                                                        | PTS   | NIVEL LOGRADO                                                                                                                                                                                                                                                                                                                  | DESCIENDE A PARCIAL SI                                                                                                         |
|-------------------------------------------------------------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| 1 · Vite + JSX                                              | 0.50  | Proyecto Vite propio que arranca y compila; JSX correcto: una raíz, className, etiquetas cerradas, datos entre llaves.                                                                                                                                                                                                         | Avisos de JSX en consola; restos de la plantilla inicial.                                                                      |
| 2 · Componentes + Props                                     | 1.00  | Un componente por archivo con responsabilidad única; props descendentes sin mutación; interfaces del dominio y props tipadas (criterio TS acotado, sección 3).                                                                                                                                                                 | Componentes extensos con responsabilidades mezcladas; props sin tipar; estructura repetida que ameritaba un componente.        |
| 3 · Renderizado condicional y de listas                     | 0.75  | key = id en toda lista; condicionales sin el error del 0 en && listado principal responsive a 360px; ítems semánticos (article, img con alt).                                                                                                                                                                                  | key por índice; un "0" visible en pantalla; maquetación rota en móvil; estructura sin semántica.                               |
| 4 · Eventos + useState                                      | 1.00  | Manejadores correctamente asignados (referencia a la función, no su invocación); estado mínimo: ningún dato almacenado que pueda calcularse; derivados en el render; inmutabilidad en cada actualización.                                                                                                                      | Estados redundantes (totales almacenados aparte de la colección); mutaciones directas del estado.                              |
| 5 · useEffect + ciclo de vida                               | 1.00  | Dependencias completas y correctas; función de limpieza en todo efecto que la requiere; la colección del usuario persiste tras recargar (localStorage con try/catch y validación de lo leído).                                                                                                                                 | Efectos usados para calcular derivados; dependencias incompletas; suscripciones sin limpieza.                                  |
| 6 · API REST + estados de carga + manejo de errores         | 1.75  | Datos de una API real; los tres estados implementados: carga (skeleton), error (mensaje claro y opción de reintentar) y vacío informativo; peticiones cancelables (AbortController) sin que la cancelación dispare el estado de error; ningún catch silencioso.                                                                | Pantallas en blanco durante la carga; errores visibles solo en consola; estados de interfaz que no reflejan la situación real. |
| 7 · React Router                                            | 1.00  | Detalle accesible por URL directa (/entidad/:id); página 404 propia; navegación sin recarga; rutas operativas tras recargar en producción.                                                                                                                                                                                     | El detalle solo abre navegando desde el listado; 404 del proveedor en lugar de la propia.                                      |
| 8 · Formularios controlados + validaciones                  | 1.00  | Formulario principal controlado (value/onChange); validación por campo con mensajes junto al input; envío bloqueado ante errores; cada input con su label y operable por teclado.                                                                                                                                              | Solo validación nativa del navegador; mensajes genéricos; inputs sin label.                                                    |
| 9 · Custom Hooks + Context + estado global                  | 1.00  | La colección del usuario administrada en Context (sin cadenas de props); al menos un custom hook propio con uso efectivo.                                                                                                                                                                                                      | Context declarado pero con props aún en cadena; hooks propios sin uso real.                                                    |
| 10 · JWT + sesión + rutas protegidas + variables de entorno | 1.50  | Autenticación real contra la API; el token se obtiene y <b>se utiliza</b> (Authorization: Bearer); sesión persistente con expiración; rutas protegidas que redirigen y retornan tras el login; configuración en .env, sin credenciales ni URLs codificadas en el fuente; los errores de autenticación se comunican al usuario. | Token almacenado pero nunca enviado; sesión sin expiración; ruta protegida que no retorna al destino original.                 |
| 11 · Optimización de componentes                            | 0.25  | memo o lazy aplicados con criterio y con evidencia (chunks separados en el build).                                                                                                                                                                                                                                             | Uso indiscriminado de memo sin justificación técnica.                                                                          |
| 12 · Despliegue en Vercel                                   | 0.25  | Aplicación operativa desde cualquier dispositivo, con rewrite de SPA configurado (toda ruta soporta recarga).                                                                                                                                                                                                                  | Desplegada, pero las rutas internas devuelven 404 al recargar.                                                                 |
| Total                                                       | 11.00 |                                                                                                                                                                                                                                                                                                                                |                                                                                                                                |

5. Bonus de arquitectura (hasta +1.00; tope del componente: 11)

| BONUS                                        | PTS   | CONDICIONES                                                                                                                                                                                                                                                                                                                                                                                             |
|----------------------------------------------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Arquitectura hexagonal puertos y adaptadores | +1.00 | Dominio separado de React y de la infraestructura: lógica de negocio en módulos puros (sin imports de React), adaptadores para API y almacenamiento detrás de interfaces propias, y la interfaz de usuario como capa de entrega. Se otorga completo únicamente si el estudiante sustenta el propósito de cada capa durante la presentación; una estructura de carpetas sin separación real no califica. |
| Funcionalidad propia alternativa al anterior | +0.50 | Una funcionalidad, integración o sección no cubierta en el curso, operativa y sustentable.                                                                                                                                                                                                                                                                                                              |

6. Sustentación (3 pts): 10 minutos de exposición + 5 de preguntas

- **Exposición (10 min):** demostración de la aplicación en producción, recorriendo el flujo completo (listado, búsqueda, detalle, colección, sesión y formulario), y defensa de una decisión de diseño propia: el estudiante elige una pieza, estructura o solución que decidió él mismo y expone sus razones.
- **Ronda de preguntas (5 min):** incluye las preguntas del banco conceptual (sección 7). El docente puede solicitar además una modificación acotada en vivo (por ejemplo: agregar un campo al contrato y propagarlo); en ese caso se evalúa el dominio del propio código, no la perfección del cambio.

7. Dominio conceptual (3 pts): preguntas de muestra

Durante la ronda de preguntas se formulan dos o tres, seleccionadas al azar, que se responden señalando el propio código. El banco completo mantiene este mismo estilo:

1. Señale en su App un **dato derivado**. ¿Por qué no es un `useState`? (estado mínimo)
2. ¿Por qué su función que agrega a la colección devuelve un array nuevo en lugar de usar `push`? (inmutabilidad y referencias)
3. ¿Qué ocurriría si su listado filtrado usara el índice como `key`? (identidad de listas)
4. Muestre la **función de limpieza** de uno de sus efectos. ¿Qué sucede si se elimina? (ciclo de vida)
5. Ponga su aplicación en modo Offline (DevTools). ¿Qué ve el usuario y por qué ese comportamiento es correcto? (manejo de errores)
6. Muestre dónde viaja su token después del login. ¿Qué sucede cuando expira? (JWT y sesión)
7. ¿Por qué su colección está en `Context` y no descendiendo por props? ¿En qué caso no usaría `Context`? (estado global)
8. ¿Qué contiene su `.env` y por qué no se incluye en el repositorio? (variables de entorno)

8. Condiciones generales

- **Originalidad:** el proyecto es individual, con tema, nombre e identidad propios. Las demostraciones del curso sirven de referencia, no de plantilla.
- **Uso de documentación e IA:** se permite como apoyo del aprendizaje. Todo lo entregado debe poder explicarse y modificarse en vivo durante la sustentación; el banco conceptual y la modificación en vivo verifican precisamente esto.
- **Estándar de entrega:** `npm run build` sin errores, commit y push, en todas las entregas del bloque sin excepción.